hashlock.

Security Audit

Opengradient 2nd (dApp)

July 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

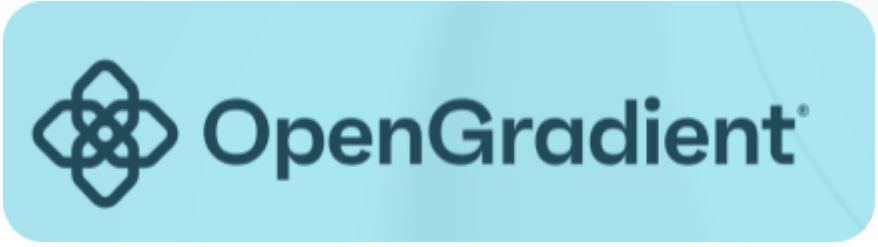
Executive Summary

The Opengradient team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

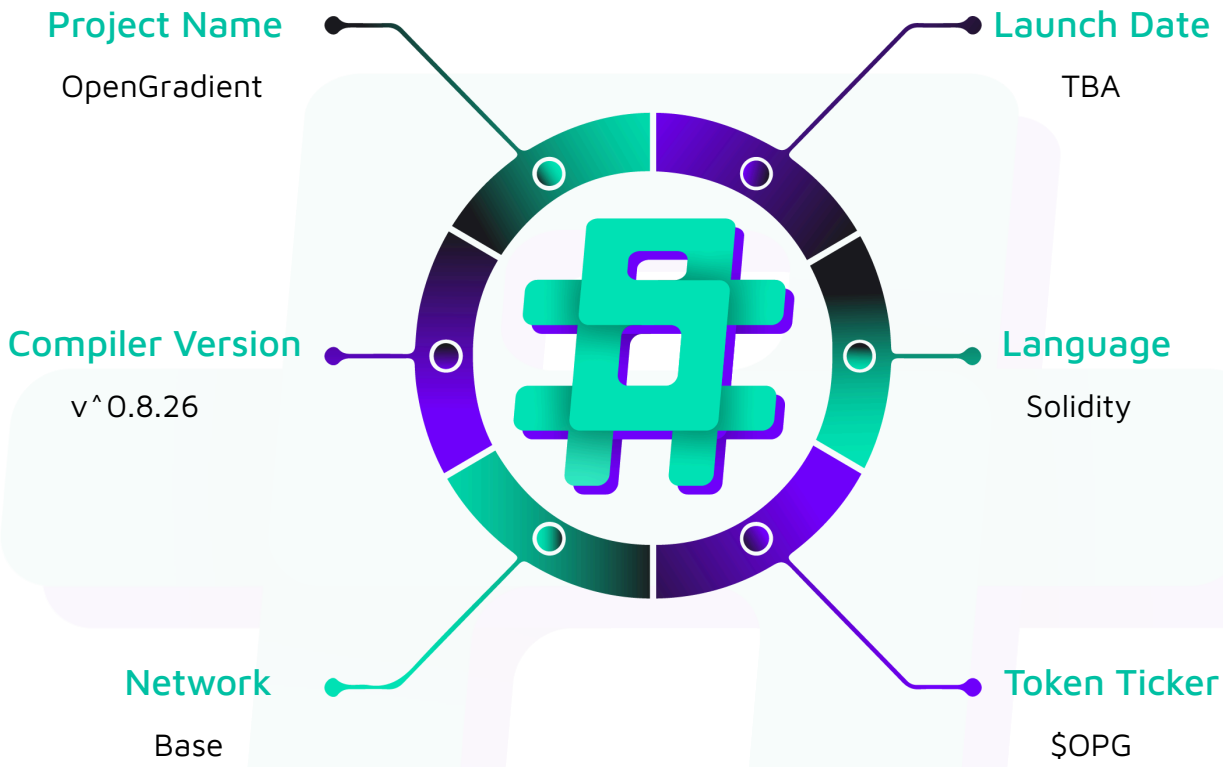
Project Context

OpenGradient is a decentralized AI infrastructure platform designed to enable secure, verifiable hosting, execution, and deployment of artificial intelligence models and applications. It provides an end-to-end network where developers can upload models to a decentralized repository, run AI inference with cryptographic guarantees, and build composable AI agents and applications that seamlessly integrate with both Web2 and blockchain environments. The architecture emphasizes privacy, transparency, and decentralization through verifiable compute and persistent context memory, empowering developers to leverage scalable AI capabilities without relying on centralized providers.

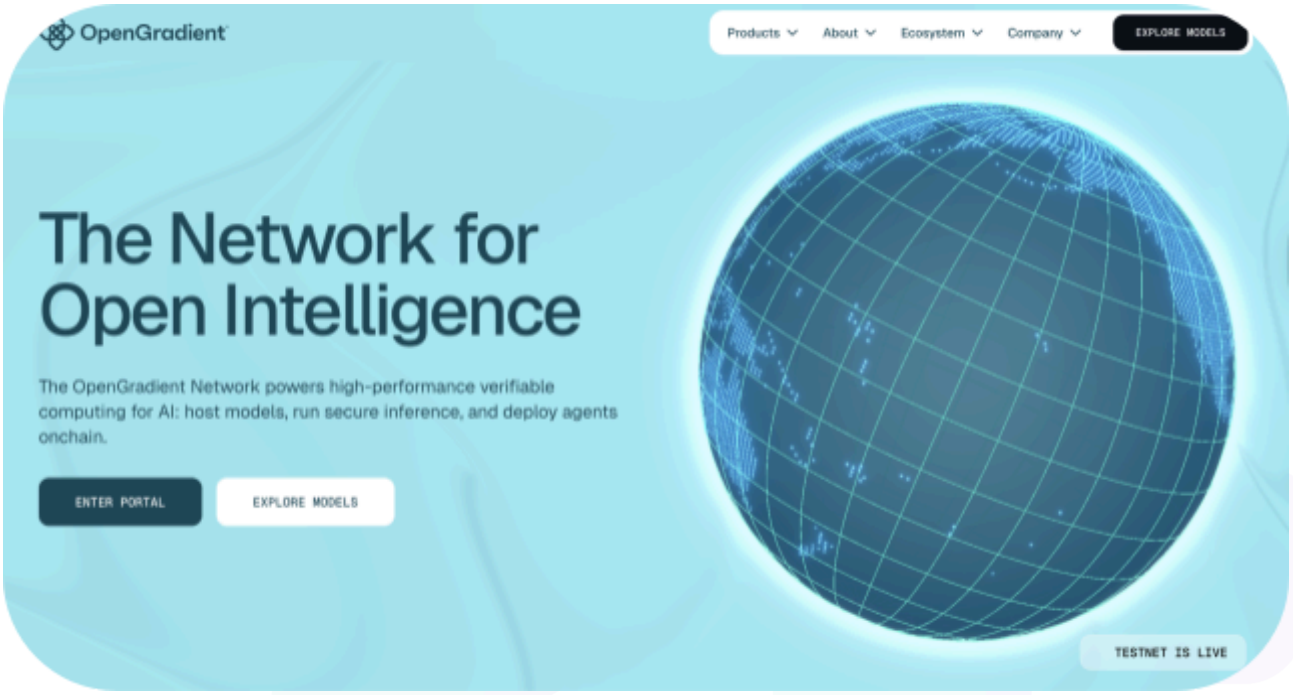
Project Name: OpenGradient
Project Type: dApp
Compiler Version: ^0.8.26
Website: https://www.opengradient.ai/
Logo:

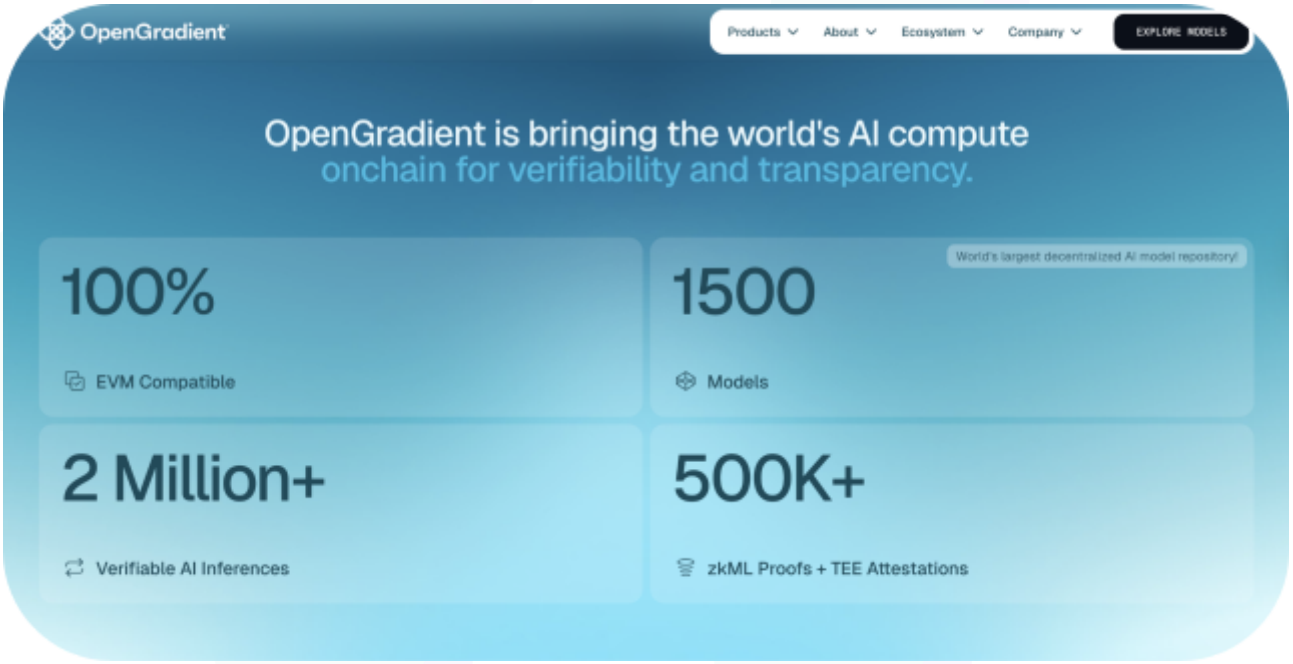


Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Opengradient project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Opengradient Smart Contracts**
Network	**Base**
Language	**Solidity**
Audit Date	**July, 2026**
Contract 1	OPGStakingVault.sol
Audited GitHub Commit Hash	24eb80539a88fdcd1bfa02b0863aa34fce2284f8
Fix Review GitHub Commit Hash	f7252ce772318accb8e35b063a05cac4ee5e735c

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

5 Low severity vulnerabilities

7 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

opg-vault-audit/OPGStakingVault.sol
• Allows the `owner` (`Ownable2Step`) to:
  ○ Pause new deposits and mints via `pauseDeposits` (withdrawals and redemptions remain open).
  ○ Re-enable deposits and mints via `unpauseDeposits`.
  ○ Hand over ownership through the two-step propose/accept flow (`transferOwnership` then `acceptOwnership`), so a mistyped or dead address cannot strand the role.
  ○ Relinquish ownership via the inherited one-step `renounceOwnership`.
• Allows the `admin` (`DEFAULT_ADMIN_ROLE`) to:
  ○ Grant and revoke `FUNDER_ROLE` and `DEFAULT_ADMIN_ROLE` via the inherited `grantRole` / `revokeRole`, delegating reward funding to a separate distributor.
• Allows the `funder` (`FUNDER_ROLE`) to:
  ○ Fund a reward via `fundReward`: pull OPG from the caller and stream it into the share price linearly over `REWARD_DURATION` (30 days), rolling any still-unvested remainder of the previous stream into the new one.
• Allows users to:
	Contract achieves this functionality.

○ Stake OPG and receive `sOPG` shares via `deposit` / `mint` (blocked while deposits are paused). ○ Exit at any time by burning `sOPG` for OPG via `withdraw` / `redeem` (never paused). ○ Read the still-locked (unvested) portion of the current reward stream via `lockedRewards`. ○ Read the vested portion of the current reward stream via `streamedRewards`. ○ Read the assets the vault is accountable for (OPG balance minus unvested rewards) via `totalAssets`. ○ Read the deposit/mint caps via `maxDeposit` / `maxMint`, which return `0` while paused. ○ Hold and transfer their `sOPG` shares as a standard ERC-20 (24 decimals: 18 underlying + a 6-decimal virtual-share offset).	

Code Quality

This audit scope involves the smart contracts of the Opengradient project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Opengradient project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] OPGStakingVault#fundReward - Rewards funded to an empty or dust-level vault are stranded behind virtual shares or disproportionately captured

Description

`fundReward` (`OPGStakingVault.sol:124-139`) pulls OPG and begins a 30-day linear vesting stream without verifying that any real shares exist, i.e. there is no `require(totalSupply() > 0)`. `totalAssets()` (`OPGStakingVault.sol:116`) grows as the stream vests regardless of the share supply, while the OpenZeppelin virtual-share defense (`_decimalsOffset()` returns `6`, i.e. `1e6` phantom shares) only protects a vault that already holds meaningful real assets. Two failure paths exist in an un-seeded vault: (1) if a reward vests while `totalSupply() == 0`, the vested OPG is permanently bound to the `1e6` virtual shares and is never redeemable by any subsequent depositor; (2) if a dust position exists (a 1-wei deposit into an empty vault mints `1e6` shares, exactly matching the virtual shares), that dust holder captures roughly 50% of the reward stream — approaching ~100% for a slightly larger deposit — at negligible cost. The NatSpec (`OPGStakingVault.sol:28-29`) recommends seeding a first deposit at deployment but the code never enforces it.

Vulnerability Details

Both failure paths are confined to an un-seeded / launch-window vault: an established vault with real TVL is unaffected and no existing user's principal is ever at risk. Root cause: `fundReward` lacks a `totalSupply()` guard and deployment does not mint a permanent seed / dead-shares position, so reward vesting can occur against only the `1e6` virtual shares. `totalAssets()` reads the raw balance minus locked rewards and is share-supply-agnostic:

```
function totalAssets() public view override returns (uint256) {
    return IERC20(asset()).balanceOf(address(this)) - lockedRewards();
```

```
}
```

The virtual-share offset is fixed at `6`:

```
function _decimalsOffset() internal pure override returns (uint8) {
    return DECIMALS_OFFSET; // 6 -> 1e6 virtual shares
}
```

Stranding math: after a `100e18` reward fully vests into a zero-share vault, Alice deposits `100e18` and receives `shares = 100e18 * (0 + 1e6) / (100e18 + 1) ~= 1e6`. Her redeem value is `1e6 * (200e18 + 1) / (1e6 + 1e6) = 100e18` — exactly her principal. The reward's fraction stays locked to the virtual shares at ratio `1e6 / (supply + 1e6)` for every future depositor and is irrecoverable.

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import {OPGStakingVault} from "../src/OPGStakingVault.sol";
import {ERC20Mock} from "@openzeppelin/contracts/mocks/token/ERC20Mock.sol";

contract L01_EmptyVaultRewardTest is Test {
    OPGStakingVault vault;
    ERC20Mock opg;
    address owner  = makeAddr("owner");   // holds FUNDER_ROLE
    address alice  = makeAddr("alice");   // honest late depositor
    address dust   = makeAddr("dust");    // dust attacker

    function setUp() public {
        opg = new ERC20Mock();
        vault = new OPGStakingVault(opg, owner); // NO seed deposit at deployment
    }

    // Path 1: reward vests into a zero-share vault -> permanently stranded.
    function test_RewardStrandedInEmptyVault() public {
        // Fund reward while totalSupply() == 0 (missing guard lets this through).
```

```
        opg.mint(owner, 100e18);
        vm.startPrank(owner);
        opg.approve(address(vault), 100e18);
        vault.fundReward(100e18);
        vm.stopPrank();

        // Let the whole 30-day stream vest.
        vm.warp(block.timestamp + 30 days);
        assertEq(vault.totalSupply(), 0);
        assertEq(vault.totalAssets(), 100e18); // reward fully vested, zero real
shares

        // Alice deposits her own 100e18.
        opg.mint(alice, 100e18);
        vm.startPrank(alice);
        opg.approve(address(vault), 100e18);
        uint256 shares = vault.deposit(100e18, alice);
        vm.stopPrank();

        // shares ~= 1e6, matching the 1e6 virtual shares.
        assertApproxEqAbs(shares, 1e6, 1);

        // Alice redeems everything she holds.
        vm.prank(alice);
        uint256 out = vault.redeem(shares, alice, alice);

        // She recovers only her own deposit; the 100e18 reward is stranded forever
        // behind the virtual shares (no address can ever redeem it).
        assertApproxEqAbs(out, 100e18, 1e6);
        assertGt(opg.balanceOf(address(vault)), 99e18); // reward still trapped
    }

    // Path 2: 1-wei dust deposit captures ~50% of the reward for negligible cost.
    function test_DustHolderCapturesHalfTheReward() public {
        // Attacker seeds 1 wei into the empty vault -> mints exactly 1e6 shares.
        opg.mint(dust, 1);
        vm.startPrank(dust);
        opg.approve(address(vault), 1);
```

```
        uint256 dustShares = vault.deposit(1, dust);
        vm.stopPrank();
        assertEq(dustShares, 1e6); // == virtual shares

        // Funder streams 100e18; let it vest.
        opg.mint(owner, 100e18);
        vm.startPrank(owner);
        opg.approve(address(vault), 100e18);
        vault.fundReward(100e18);
        vm.stopPrank();
        vm.warp(block.timestamp + 30 days);

        // Attacker redeems: 1e6 * (100e18) / (1e6 + 1e6) ~= 50e18 for a 1-wei cost.
        vm.prank(dust);
        uint256 out = vault.redeem(dustShares, dust, dust);
        assertApproxEqAbs(out, 50e18, 1e12);
    }
}
```

Impact

Permanent, irreversible loss of the funder's reward OPG when it is funded to a zero-share vault, and disproportionate reward capture by a dust holder in a near-empty vault. The exposure is confined to an un-seeded / launch-window vault whose funding timing the trusted funder controls; established vaults and existing user principal are unaffected.

Recommendation

Add a share-supply guard to `fundReward` and, more importantly, enforce seeding at deployment rather than only documenting it — mint a permanent dead-shares position (a small deposit to a burn/hold address in or immediately after the constructor) so `totalSupply()` is always meaningfully non-zero before any reward is funded. The seed is the primary fix (it also closes the dust-capture window); the `require` is defense-in-depth.

```
function fundReward(uint256 amount) external onlyRole(FUNDER_ROLE) {
```

```
     require(amount > 0, "ZERO_REWARD");
+    require(totalSupply() > 0, "NO_SHARES");

     uint256 leftover = lockedRewards();
     IERC20(asset()).safeTransferFrom(msg.sender, address(this), amount);
     ...
 }
```

```
// Seed permanent dead shares at/after construction so totalSupply() is never 0:
// e.g. deposit a small amount and send the shares to a hold/burn address.
opg.approve(address(vault), SEED);
vault.deposit(SEED, address(0xdead));
```

Status

Resolved

[L-02] OPGStakingVault#fundReward - Records the requested amount, not the delivered amount, causing accounting desync and totalAssets() underflow DoS under fee-on-transfer / rebasing tokens

Description

TOKEN-CONTINGENT. `fundReward` pulls `amount` via `safeTransferFrom` (`OPGStakingVault.sol:128`) but sets `streamTotal = leftover + amount` (`OPGStakingVault.sol:130,134`) from the caller-supplied argument instead of the measured balance delta. Immediately after funding, `lockedRewards()` returns the full `streamTotal` (the `block.timestamp <= start` branch, `OPGStakingVault.sol:101`), and `totalAssets()` (`OPGStakingVault.sol:116`) computes `balanceOf(this) - lockedRewards()` with Solidity 0.8 checked subtraction. If the asset took a transfer fee (or rebased down), the vault receives `amount*(1-fee)` but accounts the full `amount` as locked; whenever accounted-locked exceeds the real balance, `totalAssets()` underflows and reverts, bricking `deposit/mint/withdraw/redeem` and every `preview/max` call until enough of the stream vests. When it does not underflow, the fee is silently absorbed by existing shareholders.

Vulnerability Details

Under the stated trust model OPG is a standard 18-decimal, non-fee, non-rebasing ERC-20, in which case transfers are exact, the invariant holds, and there is no impact. The issue only materializes if the vault were ever pointed at a fee-on-transfer or rebasing asset — a token-contingent integration hazard. Root cause: trust-on-input accounting — `streamTotal` derives from the requested `amount` rather than the post-minus-pre balance delta, breaking the by-construction invariant `lockedRewards() <= balanceOf(this)` (`OPGStakingVault.sol:113-114`) for any token whose transfer does not credit the exact amount.

```
uint256 leftover = lockedRewards();
IERC20(asset()).safeTransferFrom(msg.sender, address(this), amount);

uint256 newTotal = leftover + amount;   // <-- uses requested amount, not delta
...
streamTotal = newTotal;
```

Right after funding, `lockedRewards()` returns the whole total, so `totalAssets()` subtracts a locked figure larger than the real balance:

```
if (block.timestamp <= start) return total;   // lockedRewards() == streamTotal
...
return IERC20(asset()).balanceOf(address(this)) - lockedRewards(); // checked
underflow
```

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import {OPGStakingVault} from "../src/OPGStakingVault.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

// 2% fee-on-transfer token to model the token-contingent hazard.
contract FeeToken is ERC20 {
    uint256 public constant FEE_BPS = 200; // 2%
    constructor() ERC20("Fee", "FEE") {}
    function mint(address to, uint256 amt) external { _mint(to, amt); }
    function _update(address from, address to, uint256 value) internal override {
        if (from != address(0) && to != address(0)) {
            uint256 fee = value * FEE_BPS / 10_000;
            super._update(from, to, value - fee);
            super._update(from, address(0xFEE), fee); // fee siphoned off
        } else {
            super._update(from, to, value);
        }
    }
}

contract L02_FeeOnTransferDoSTest is Test {
    OPGStakingVault vault;
    FeeToken fee;
    address owner = makeAddr("owner"); // FUNDER_ROLE
    address alice = makeAddr("alice");
```

```
    function setUp() public {
        fee = new FeeToken();
        vault = new OPGStakingVault(fee, owner); // vault pointed at a fee token
    }

    function test_FundRewardUnderflowsTotalAssets() public {
        // Near-empty vault: principal 0, streamTotal 0.
        fee.mint(owner, 10_000e18);
        vm.startPrank(owner);
        fee.approve(address(vault), 10_000e18);

        // Requests 10_000e18; vault only receives 9_800e18, but records 10_000e18.
        vault.fundReward(10_000e18);
        vm.stopPrank();

        assertEq(fee.balanceOf(address(vault)), 9_800e18);
        assertEq(vault.streamTotal(), 10_000e18); // accounted > real balance

        // lockedRewards() == 10_000e18 right after funding -> totalAssets()
underflows.
        vm.expectRevert(); // arithmetic underflow (checked math)
        vault.totalAssets();

        // Every 4626 entry/exit path is bricked because they all call totalAssets().
        fee.mint(alice, 1e18);
        vm.startPrank(alice);
        fee.approve(address(vault), 1e18);
        vm.expectRevert();
        vault.deposit(1e18, alice);
        vm.stopPrank();

        // Recovers only once ~2% of the 30-day window vests (~14.4h), when
        // lockedRewards() drops below the real 9_800e18 balance.
        vm.warp(block.timestamp + 15 hours);
        vault.totalAssets(); // no revert now
    }
}
```

Impact

None under the assumed standard OPG. If the asset were fee-on-transfer / rebasing: temporary full DoS of every ERC-4626 entry/exit path (until roughly fee% of the 30-day window elapses) when vault principal is smaller than the fee shortfall, otherwise a silent value leak subsidized by current shareholders. No attacker profit; `fundReward` is `FUNDER_ROLE`-gated.

Recommendation

Use the measured balance delta: read `balanceOf(this)` before and after the `safeTransferFrom`, compute `received = after - before`, require it non-zero, and use `received` for `newTotal` and the `RewardFunded` event. Alternatively, document as a hard integration constraint that the vault must never be deployed against a fee-on-transfer or rebasing asset.

```
function fundReward(uint256 amount) external onlyRole(FUNDER_ROLE) {
     require(amount > 0, "ZERO_REWARD");

     uint256 leftover = lockedRewards();
-    IERC20(asset()).safeTransferFrom(msg.sender, address(this), amount);
+    uint256 balBefore = IERC20(asset()).balanceOf(address(this));
+    IERC20(asset()).safeTransferFrom(msg.sender, address(this), amount);
+    uint256 received = IERC20(asset()).balanceOf(address(this)) - balBefore;
+    require(received > 0, "NO_TOKENS_RECEIVED");

-    uint256 newTotal = leftover + amount;
+    uint256 newTotal = leftover + received;
     uint64 end = uint64(block.timestamp + REWARD_DURATION);
     streamTotal = newTotal;
     streamStart = uint64(block.timestamp);
     streamEnd = end;

-    emit RewardFunded(msg.sender, amount, newTotal, end);
+    emit RewardFunded(msg.sender, received, newTotal, end);
 }
```

Status

Resolved

[L-03] OPGStakingVault#renounceOwnership - Renouncing ownership while deposits are paused permanently bricks deposits

Description

`unpauseDeposits()` (`OPGStakingVault.sol:148`, `onlyOwner`) is the only path to clear a paused state, and `_deposit` (`OPGStakingVault.sol:163-169`) is `whenNotPaused`. The vault inherits `Ownable2Step`, but `Ownable2Step` does NOT override the base `Ownable.renounceOwnership()`: that function remains `public onlyOwner` and sets the owner to `address(0)` in a single step, with none of the propose/accept protection that guards `transferOwnership`. If the owner pauses deposits and then renounces ownership, no account can ever satisfy `onlyOwner` again, `unpauseDeposits()` becomes permanently uncallable, and deposits/mints are disabled forever (`maxDeposit`/`maxMint` return `0` while paused). Withdrawals and redemptions are never paused, so existing stakers can still exit — no principal is lost, but the vault can never accept new stake.

Vulnerability Details

A trusted owner performing a legitimate-but-mistimed action (renouncing for decentralization while a pause is active) makes the paused state terminal. There is no fund loss and no attacker, but the consequence is an irreversible denial of the deposit lifecycle with no recovery path. Root cause: `Ownable2Step` leaves the one-step `Ownable.renounceOwnership()` exposed and un-overridden while the entire pause lifecycle depends on a live owner to unpause. The only unpause path is owner-gated:

```
function unpauseDeposits() external onlyOwner {
    _unpause();
}
```

and the deposit path is hard-gated by the pause modifier:

```
function _deposit(address caller, address receiver, uint256 assets, uint256 shares)
    internal
    override
    whenNotPaused
{
    super._deposit(caller, receiver, assets, shares);
```

```
}
```

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import {OPGStakingVault} from "../src/OPGStakingVault.sol";
import {ERC20Mock} from "@openzeppelin/contracts/mocks/token/ERC20Mock.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract L03_RenounceWhilePausedTest is Test {
    OPGStakingVault vault;
    ERC20Mock opg;
    address owner = makeAddr("owner");
    address alice = makeAddr("alice");

    function setUp() public {
        opg = new ERC20Mock();
        vault = new OPGStakingVault(opg, owner);
        opg.mint(alice, 100e18);
        vm.prank(alice);
        opg.approve(address(vault), type(uint256).max);
    }

    function test_RenounceWhilePausedBricksDepositsForever() public {
        // 1. Owner pauses deposits.
        vm.prank(owner);
        vault.pauseDeposits();
        assertTrue(vault.paused());
        assertEq(vault.maxDeposit(alice), 0);

        // 2. Owner renounces via the inherited one-step Ownable.renounceOwnership().
        vm.prank(owner);
        vault.renounceOwnership();
        assertEq(vault.owner(), address(0));
```

```
        // 3. Deposits revert permanently (paused, maxDeposit == 0).
        vm.prank(alice);
        vm.expectRevert();
        vault.deposit(1e18, alice);

        // 4. Recovery is impossible: unpause is onlyOwner and no owner exists.
        vm.prank(owner);
        vm.expectRevert(
            abi.encodeWithSelector(Ownable.OwnableUnauthorizedAccount.selector,
owner)
        );
        vault.unpauseDeposits();

        // 5. Existing stakers can still exit (withdraw/redeem are never paused),
        //    but no new stake can ever enter.
    }
}
```

Impact

Permanent, irreversible denial of the deposit/mint core function if ownership is renounced while paused. No fund loss (exits remain open) and no attacker required, but the vault becomes a one-way, no-new-stake contract with zero recovery path.

Recommendation

Override `renounceOwnership()` to revert, or at minimum require `!paused()` so ownership cannot be dropped while the vault is in a state only the owner can exit. If renouncing must remain supported, decouple unpause from the owner (e.g. also allow `DEFAULT_ADMIN_ROLE` to unpause) so a single renounce cannot strand the lifecycle.

```
// Option A: disable renounce entirely.
function renounceOwnership() public view override onlyOwner {
    revert("RENOUNCE_DISABLED");
}

// Option B: block renounce while the vault is paused.
function renounceOwnership() public override onlyOwner {
```

```
    require(!paused(), "PAUSED");
    super.renounceOwnership();
}
```

Status

Resolved

[L-04] OPGStakingVault#renounceRole - Sole DEFAULT_ADMIN_ROLE holder can renounce, irreversibly freezing role administration and disabling reward funding

Description

The constructor (`OPGStakingVault.sol:83-86`) grants `DEFAULT_ADMIN_ROLE` and `FUNDER_ROLE` to a single address, and the contract inherits plain `AccessControl` (not `AccessControlDefaultAdminRules`). `DEFAULT_ADMIN_ROLE` is its own admin, `renounceRole`/`revokeRole` enforce no minimum holder count, and `_grantRole` is only ever called in the constructor. If the sole admin renounces `DEFAULT_ADMIN_ROLE`, no code path can ever grant or revoke any role again: `FUNDER_ROLE` administration gates on the now-empty admin role, and `Ownable2Step` ownership is explicitly independent (NatSpec `OPGStakingVault.sol:41-43`) and confers no AccessControl power, so transferring ownership cannot recover it. Once existing `FUNDER_ROLE` holders are also renounced or their keys lost, `fundReward` (`OPGStakingVault.sol:124`, `onlyRole(FUNDER_ROLE)`) is permanently unreachable from every address — the reward program is dead with no recovery, while `deposit`/`withdraw`/`redeem` keep working. Even renouncing the admin alone is already irreversible: `FUNDER_ROLE` membership is frozen, so a later funder key loss or compromise cannot be remediated.

Vulnerability Details

Root cause: a single admin at deployment combined with unguarded inherited `renounceRole`/`revokeRole` — no last-admin floor is enforced and no post-constructor grant path exists, so losing the last `DEFAULT_ADMIN_ROLE` holder is unrecoverable by construction.

```
constructor(IERC20 asset_, address initialOwner) ERC20("Staked OPG", "sOPG")
ERC4626(asset_) Ownable(initialOwner) {
    _grantRole(DEFAULT_ADMIN_ROLE, initialOwner);
    _grantRole(FUNDER_ROLE, initialOwner);
}
```

This is distinct from L-03 (`Ownable.renounceOwnership` bricking the pause lifecycle) and from the stale-roles concern in Q-02 (excess role holders retained after an ownership handover — the opposite failure mode). Structurally it is the same class of irreversible

foot-gun as L-03: a plausible "lock-down / decentralization" action with a permanent, zero-recovery consequence, no attacker, and no user fund risk — hence Low.

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import {OPGStakingVault} from "../src/OPGStakingVault.sol";
import {ERC20Mock} from "@openzeppelin/contracts/mocks/token/ERC20Mock.sol";

contract L04_AdminRenounceFreezesRolesTest is Test {
    OPGStakingVault vault;
    ERC20Mock opg;
    address owner    = makeAddr("owner");    // sole admin + funder
    address newFunder= makeAddr("newFunder");
    address rescuer  = makeAddr("rescuer");

    bytes32 ADMIN;
    bytes32 FUNDER;

    function setUp() public {
        opg = new ERC20Mock();
        vault = new OPGStakingVault(opg, owner);
        ADMIN  = vault.DEFAULT_ADMIN_ROLE();
        FUNDER = vault.FUNDER_ROLE();
    }

    function test_AdminRenounceMakesFundingUnrecoverable() public {
        // Step 1: sole admin renounces (plausible lock-down action).
        vm.prank(owner);
        vault.renounceRole(ADMIN, owner);

        // Role administration is now frozen forever.
        vm.prank(owner);
        vm.expectRevert(); // AccessControlUnauthorizedAccount
        vault.grantRole(FUNDER, newFunder);
```

```
        // Step 2: last funder is also renounced (or key lost).
        vm.prank(owner);
        vault.renounceRole(FUNDER, owner);

        // fundReward is now permanently unreachable from EVERY address.
        opg.mint(owner, 100e18);
        vm.startPrank(owner);
        opg.approve(address(vault), 100e18);
        vm.expectRevert(); // AccessControlUnauthorizedAccount(owner, FUNDER)
        vault.fundReward(100e18);
        vm.stopPrank();

        // Ownership transfer does NOT recover it (owner holds no AccessControl
power).
        vm.prank(owner);
        vault.transferOwnership(rescuer);
        vm.prank(rescuer);
        vault.acceptOwnership();
        vm.prank(rescuer);
        vm.expectRevert();
        vault.grantRole(FUNDER, rescuer);

        // deposit/withdraw/redeem still work; only the reward program is dead.
    }
}
```

Impact

Irreversible loss of role administration; in the worst case `fundReward` can never succeed again, permanently killing the reward-streaming program. No user principal is at risk — staking, withdrawal, and redemption remain fully functional — and no attacker profits.

Recommendation

Inherit OpenZeppelin `AccessControlDefaultAdminRules` (a single, transfer-delayed, non-freezable default admin), or override `renounceRole`/`revokeRole` to revert when the action would remove the last `DEFAULT_ADMIN_ROLE` holder. At minimum, deploy with a

multisig/timelock as admin (or two independent admin addresses) and add a runbook rule that `DEFAULT_ADMIN_ROLE` is never renounced without a successor already granted.

```
import {AccessControlDefaultAdminRules}
    from
"@openzeppelin/contracts/access/extensions/AccessControlDefaultAdminRules.sol";

// Replace `AccessControl` with `AccessControlDefaultAdminRules` in the inheritance
list
// and pass an initial delay + admin in the constructor, e.g.:
//   AccessControlDefaultAdminRules(3 days, initialOwner)
// This makes the default admin single, transfer-delayed, and impossible to leave
empty.
```

Status

Resolved

[L-05] OPGStakingVault - No token-rescue path leaves non-asset ERC-20s sent to the vault permanently unrecoverable

Description

The vault's complete external/public surface is `fundReward`, `pauseDeposits`, `unpauseDeposits`, and the inherited ERC-4626 `deposit`/`mint`/`withdraw`/`redeem` — none of which can move a foreign (non-OPG) ERC-20 out of the contract, and there is no sweep/rescue/recover function. Any non-asset token accidentally transferred to the vault address (user mistake, airdrop, misrouted transfer) is therefore locked forever; the contract is non-upgradeable, so the lockup is unremediable post-deployment. Note that a raw OPG transfer sent directly to the vault immediately counts as shareholder value and cannot be clawed back; this is mechanically true but explicitly documented as intended behavior (NatSpec `OPGStakingVault.sol:34-36`), benefits shareholders, and is a design choice rather than a defect, so it is out of scope for this finding.

Vulnerability Details

Root cause: the contract inherits `ERC4626`/`Ownable2Step`/`AccessControl`/`Pausable` but never adds a privileged token-recovery function, so it can custody arbitrary ERC-20 balances with no code path capable of transferring them out. Foreign tokens do not affect vault accounting — `totalAssets()` reads only the `asset()` balance:

```
function totalAssets() public view override returns (uint256) {
    return IERC20(asset()).balanceOf(address(this)) - lockedRewards();
}
```

so solvency and share pricing are untouched; the loss falls solely on whoever misdirects a transfer. Permanent and irreversible with no remediation possible qualifies it as a Low foot-gun rather than QA; the absence of any attacker path or fund risk to vault users caps it below Medium.

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
import "forge-std/Test.sol";
import {OPGStakingVault} from "../src/OPGStakingVault.sol";
import {ERC20Mock} from "@openzeppelin/contracts/mocks/token/ERC20Mock.sol";

contract L05_NoRescueTest is Test {
    OPGStakingVault vault;
    ERC20Mock opg;   // the vault asset
    ERC20Mock usdc;  // an unrelated token misrouted to the vault
    address owner = makeAddr("owner");
    address alice = makeAddr("alice");

    function setUp() public {
        opg = new ERC20Mock();
        usdc = new ERC20Mock();
        vault = new OPGStakingVault(opg, owner);
    }

    function test_ForeignTokenIsLockedForever() public {
        // 1. Alice fat-fingers 10,000 USDC to the vault address.
        usdc.mint(alice, 10_000e6);
        vm.prank(alice);
        usdc.transfer(address(vault), 10_000e6);
        assertEq(usdc.balanceOf(address(vault)), 10_000e6);

        // 2. Vault accounting is untouched (totalAssets never reads USDC).
        assertEq(vault.totalAssets(), 0);

        // 3. There is no function on the vault that can move USDC out.
        //    fundReward pulls OPG in only; pause/unpause move nothing;
        //    withdraw/redeem pay out asset() == OPG only.
        //    The owner has no callable path to return the 10,000 USDC.

        // A hypothetical rescueToken(usdc, alice, 10_000e6) (see recommendation)
        // would let the owner return it, while still refusing to move OPG.
        assertEq(usdc.balanceOf(address(vault)), 10_000e6); // still trapped
    }
}
```

Impact

Non-OPG tokens mistakenly sent to the vault are irrecoverably locked. No loss of protocol funds, no impact on vault solvency or share accounting, and no attacker profit — the loss falls on the third party who misdirects the transfer.

Recommendation

Add an owner-restricted rescue function that explicitly refuses the vault's own `asset()`. Refusing `asset()` is essential — allowing it would let the owner withdraw user deposits / locked rewards and break the solvent-by-construction invariant.

```
function rescueToken(IERC20 token, address to, uint256 amount) external onlyOwner {
    require(address(token) != asset(), "CANNOT_RESCUE_ASSET");
    require(to != address(0), "ZERO_ADDR");
    token.safeTransfer(to, amount);
}
```

Status

Acknowledged

QA

[Q-01] OPGStakingVault#_decimalsOffset - sOPG share token reports 24 decimals (18 underlying + 6 offset), an undocumented integration/display caveat

Description

OZ `ERC4626.decimals()` returns `_underlyingDecimals + _decimalsOffset()`. With the standard 18-decimal OPG and the vault's `_decimalsOffset()` override returning 6 (`DECIMALS_OFFSET`, `OPGStakingVault.sol:57` and `:172-174`), the `sOPG` share token's `decimals()` is 24, not 18. This is a deliberate, correct consequence of the `1e6` virtual-share anti-inflation hardening, but it is nowhere documented that the SHARE token consequently reports 24 decimals. Any front-end, indexer, or accounting integration that hardcodes 18 (or assumes shares match the underlying) will misrender `sOPG` balances by `1e6`x.

Vulnerability Details

No on-chain impact: all conversion math uses the offset consistently, cannot overflow at realistic scales (a `~1e27`-wei supply maps to `~1e33` shares, far below `uint256` max), and 24 fits the `uint8` `decimals()` return. ERC-20 `decimals` is display metadata only; spec-compliant consumers that read `decimals()` dynamically are unaffected. Purely an off-chain display/integration correctness note.

Proof of Concept

```
// underlying OPG: 18 decimals
// _decimalsOffset() == DECIMALS_OFFSET == 6
// ERC4626.decimals() == _underlyingDecimals + _decimalsOffset()
//                    == 18 + 6
decimals(); // returns 24, NOT 18
```

Impact

Off-chain only: wallets/dashboards/indexers assuming 18 decimals display `sOPG` balances 1,000,000x too large, or apply wrong scaling in derived analytics. No fund risk, no broken core function.

Recommendation

Keep the 6-decimal offset (intended inflation hardening) but document explicitly in NatSpec and integration docs that `sOPG` reports 24 decimals. Optionally a smaller offset (e.g. 3) lowers the decimals count at the cost of anti-inflation margin; no code change is required for correctness.

Status

Resolved

[Q-02] OPGStakingVault - Ownership handover does not migrate AccessControl roles; previous owner retains DEFAULT_ADMIN_ROLE / FUNDER_ROLE

Description

The vault runs two independent authority systems: `Ownable2Step` (owner gates only `pauseDeposits`/`unpauseDeposits`) and `AccessControl` (`DEFAULT_ADMIN_ROLE` + `FUNDER_ROLE` gate role administration and reward funding). The constructor grants all three authorities to `initialOwner`, but `transferOwnership`/`acceptOwnership` only rewrite the owner slot — no override links the systems. After a naive handover (`transferOwnership(B)` -> `B.acceptOwnership()`), `owner == B` (pause rights only) while `A` retains `DEFAULT_ADMIN_ROLE` and `FUNDER_ROLE`, and `B` holds neither. A complete handover requires three separate manual actions and nothing enforces the last two.

Vulnerability Details

The behavior is explicitly documented as intended in the contract's own NatSpec (`OPGStakingVault.sol:38-43`: roles "do NOT move with ownership and must be granted/revoked separately"). The ceiling of a retained ex-admin's capability is role administration plus `fundReward` — which requires supplying its own OPG and cannot move user funds — and the state is fully reversible: whoever holds `DEFAULT_ADMIN_ROLE` can revoke stale grants at any time. No fund-loss path, no irreversible brick, and any abuse requires a trusted party to act maliciously, so this is an operational handover note.

Impact

Operational only: an incomplete handover leaves the outgoing owner with role administration and funding rights. No path to withdraw, mint, drain, or pause via these roles; fully reversible by any admin.

Recommendation

Keep the decoupling but reduce operator foot-guns: (1) provide an atomic helper such as `migrateAdmin(address newAdmin)` (`onlyRole(DEFAULT_ADMIN_ROLE)`) that grants `DEFAULT_ADMIN_ROLE` + `FUNDER_ROLE` to `newAdmin` and revokes them from `msg.sender` in one call; (2) document a handover checklist (accept ownership -> grant roles to new

owner -> revoke old owner's roles); (3) monitor `RoleGranted`/`RoleRevoked` events so a lingering old-owner grant is observable.

Status

Resolved

[Q-03] OPGStakingVault#_deposit - No reentrancy guard on deposit/withdraw paths; token-contingent, only relevant if OPG had ERC-777-style transfer hooks

Description

TOKEN-CONTINGENT. The vault applies no `nonReentrant` modifiers, relying on OZ `ERC4626` internals: `_deposit` performs `safeTransferFrom(caller -> vault)` and THEN `_mint(receiver, shares)`, while `_withdraw` burns before transferring (CEI-clean). With the assumed standard ERC-20 OPG there are no transfer callbacks, so no reentrancy is reachable. Only if OPG were an ERC-777/hook token would the sender-side `tokensToSend` hook fire inside `_deposit`'s transfer — after assets are pulled but before shares are minted — giving attacker-controlled code a window against intermediate accounting.

Vulnerability Details

Under the trust model (standard 18-decimal ERC-20, no hooks, fixed known asset set in the constructor) the path is unreachable and impact is nil. `_withdraw` follows checks-effects-interactions and is not exposed even under a hook token. This is a defense-in-depth / integration-documentation note only.

Impact

Nil under the audited configuration. Hypothetically, with a hook-bearing asset, a depositor could reenter `deposit`/`mint` mid-transfer against half-updated state. No concrete profitable path exists for standard OPG.

Recommendation

Verify and document that the deployed OPG token is a plain ERC-20 with no transfer callbacks (no ERC-777 hooks, no rebasing, no reentrant transfer logic). As cheap defense-in-depth, inherit `ReentrancyGuard` and add `nonReentrant` to `fundReward` and the public `deposit`/`mint`/`withdraw`/`redeem` entry points.

Status

Resolved

[Q-04] OPGStakingVault#streamedRewards - streamedRewards() tracks only the current stream and resets to zero on every fundReward, misleading off-chain consumers

Description

`streamedRewards()` (`OPGStakingVault.sol:108-109`) returns `streamTotal - lockedRewards()`. Because `fundReward` resets `streamStart` to `block.timestamp` and sets `streamTotal = leftover + amount`, immediately after any funding call `lockedRewards()` returns the full `streamTotal` (the `block.timestamp <= start` branch, `:101`), so `streamedRewards()` reads 0. On every rollover the value collapses to zero even though no previously-vested reward has un-vested. The function is a pure view never read by `totalAssets()`, `_deposit`, or any accounting path.

Vulnerability Details

Root cause: the view is defined relative to the mutable single-stream accumulator `streamTotal`, which is overwritten on each `fundReward`; there is no persistent cumulative counter, so it can only express vesting progress inside the currently active stream. The NatSpec (`OPGStakingVault.sol:107`) correctly scopes it to "the current reward stream" — this is a naming/consumer-expectation clarity issue, not a logic defect, and the reset is a reporting artifact, not real un-vesting.

Proof of Concept

```
// t = 0    : fundReward(100e18)           -> streamTotal = 100e18, streamStart = 0
// t = 15d  : streamedRewards() == 50e18   // half of the 30-day window vested
// t = 15d  : fundReward(10e18)            // same-block top-up
//            leftover = lockedRewards() == 50e18
//            streamTotal = 50e18 + 10e18 = 60e18, streamStart = 15d
//            block.timestamp <= streamStart  ->  lockedRewards() == 60e18
// t = 15d  : streamedRewards() == 0       // collapses to 0, though 50e18 already
vested
//            totalAssets() still counts that vested 50 OPG (unchanged)
```

Impact

Purely off-chain/cosmetic: a dashboard reading `streamedRewards()` as "cumulative rewards distributed" under-reports and shows visible drops to 0 after each top-up. No effect on share price, `totalAssets`, or solvency.

Recommendation

Either (a) document explicitly that `streamedRewards()` reports the CURRENT stream's vesting progress and resets on each `fundReward`, or (b) add a monotonic accumulator (e.g. `totalRewardsFunded` incremented in `fundReward`) for lifetime dashboards, leaving `streamedRewards()` for per-stream progress.

Status

Resolved

[Q-05] OPGStakingVault#maxDeposit - previewDeposit/previewMint return non-zero quotes while deposits are paused; spec-compliant, but an integrator caveat

Description

During a pause, `maxDeposit`/`maxMint` (`OPGStakingVault.sol:153-160`) return 0 and `_deposit` is `whenNotPaused` (`:163-169`), so `deposit()`/`mint()` revert (at OZ's `ERC4626ExceededMaxDeposit`/`ERC4626ExceededMaxMint` checks). However `previewDeposit`/`previewMint` are inherited un-overridden: OZ's implementations are pure exchange-rate conversions that never consult `max*` limits or the paused flag, so during a pause `previewDeposit(x)` still returns the full normal share quote for a deposit that is guaranteed to revert.

Vulnerability Details

This divergence is mandated by EIP-4626 ("`previewDeposit` ... MUST NOT account for deposit limits like those returned from `maxDeposit` and should always act as though the deposit would be accepted"), and the contract already surfaces the paused state through the spec's designated channel (`max* == 0`). Overriding `preview*` to return 0 while paused would itself be a compliance bug. Purely an integration caveat for consumers that quote from `preview*` without gating on `max*`.

Proof of Concept

```
pauseDeposits();
maxDeposit(user);            // == 0  (paused)
maxMint(user);               // == 0  (paused)
previewDeposit(1_000e18);    // still returns ~ 1_000e18 * (totalSupply() + 1e6) /
(totalAssets() + 1)
deposit(1_000e18, user);     // reverts: ERC4626ExceededMaxDeposit(user, 1_000e18, 0)
```

Impact

No fund loss under any path. Worst case: a UI that only calls `preview*` lets a user submit a deposit during a pause; the transaction reverts cleanly, costing only gas.

Recommendation

Do NOT override `previewDeposit`/`previewMint`. Instead: (1) add a NatSpec note on `pauseDeposits`/`maxDeposit` that preview functions intentionally keep quoting while paused, per EIP-4626; (2) publish integrator guidance that UIs and contracts MUST check `maxDeposit(receiver)`/`maxMint(receiver)` (treating 0 as "deposits disabled") before quoting or executing deposits.

Status

Resolved

[Q-06] OPGStakingVault#lockedRewards - lockedRewards() rounds down, causing sub-wei reward dust to vest marginally early (solvency-safe direction)

Description

The still-locked portion (`OPGStakingVault.sol:104`) is computed as `total * (end - block.timestamp) / (end - start)` with truncating integer division, so the locked amount is understated by strictly less than 1 wei and `streamedRewards()` (`:109`) is overstated by the same amount — up to sub-1-wei of reward value counts as vested marginally before its exact linear-schedule timestamp. The discarded remainder is always `< (end - start)`, so the error is bounded below 1 wei regardless of `streamTotal`'s magnitude, and it cannot compound: every call recomputes from the stored stream fields.

Vulnerability Details

Crucially, floor rounding is the SAFE direction for the contract's stated invariant `lockedRewards() <= balanceOf(this)` (`OPGStakingVault.sol:113-114`): understating locked keeps `totalAssets()` from ever underflowing. Rounding up would have been the dangerous direction. The sub-wei is credited to `totalAssets()`/share price, marginally benefiting current shareholders; there is no attacker profit and no path to accumulate the error.

Impact

Negligible: at most just under 1 wei of OPG (`1e-18` token) treated as vested slightly ahead of the ideal continuous schedule. No solvency threat.

Recommendation

No change required — the current floor rounding is the correct, solvency-safe choice. Optionally document that `lockedRewards()` rounds down by design (`streamedRewards()` rounded up by at most 1 wei) in favor of the solvency invariant. Do NOT switch to rounding locked up, as that could push `lockedRewards()` above the real balance and revert `totalAssets()`.

Status

Acknowledged

[Q-07] OPGStakingVault#fundReward - Reward-stream window resets on every top-up (intended Synthetix-style behavior)

Description

Every `fundReward` re-locks the entire unvested leftover into a fresh 30-day window. Consequently a top-up (1) resets the vesting window to a new 30-day term and (2) re-stretches the previously remaining reward over that new term. This is intended behavior rather than a defect: the only writer of stream state is `fundReward`, gated by `onlyRole(FUNDER_ROLE)` (`OPGStakingVault.sol:124`), which is a trusted role; the behavior is fully reversible; and the re-stretch is explicitly documented (`OPGStakingVault.sol:120-122`) and matches the accepted Synthetix reward-streaming pattern. No unvested reward is ever lost, and no unprivileged actor can trigger it.

Impact

None as a security vulnerability. At most, a misconfigured or compromised `FUNDER_ROLE` holder could defer (never destroy) reward vesting — a trusted-role operational concern, not an exploit.

Recommendation

No code change is required for security. Operationally: scope `FUNDER_ROLE` to a trusted multisig/timelock, and if perpetual deferral is a concern, enforce a minimum `amount` per `fundReward` or blend the leftover on its original schedule rather than restarting it.

Status

Acknowledged

Centralisation

The Opengradient project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Opengradient project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au